

Full Length Article

Highly valued subgoal generation for efficient goal-conditioned reinforcement learning

Yao Li^a, YuHui Wang^b, XiaoYang Tan^{c,*}

^a School of Computer and Information Technology, Shanxi University, China

^b Center of Excellence in GenAI, King Abdullah University of Science and Technology, Saudi Arabia

^c College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, MIIT Key Laboratory of Pattern Analysis and Machine Intelligence, China

ARTICLE INFO

Keywords:

Goal-conditioned reinforcement learning

Sparse reward

Deep determinate policy gradient

Hindsight experience replay (HER)

ABSTRACT

Goal-conditioned reinforcement learning is widely used in robot control, manipulating the robot to accomplish specific tasks by maximizing accumulated rewards. However, the useful reward signal is only received when the desired goal is reached, leading to the issue of sparse rewards and affecting the efficiency of policy learning. In this paper, we propose a method to generate highly valued subgoals for efficient goal-conditioned policy learning, enabling the development of smart home robots or automatic pilots in our daily life. The highly valued subgoals are conditioned on the context of the specific tasks and characterized by suitable complexity for efficient goal-conditioned action value learning. The context variable captures the latent representation of the particular tasks, allowing for efficient subgoal generation. Additionally, the goal-conditioned action values regularized by the self-adaptive ranges generate subgoals with suitable complexity. Compared to Hindsight Experience Replay that uniformly samples subgoals from visited trajectories, our method generates the subgoals based on the context of tasks with suitable difficulty for efficient policy training. Experimental results show that our method achieves stable performance in robotic environments compared to baseline methods.

1. Introduction

The activities in daily life are goal-oriented, aiming to accomplish specific tasks with desired goals. Goal-conditioned reinforcement learning (GCRL) (Li et al., 2022; Nasiriany, Pong, Lin, & Levine, 2019a; Zu, He, Yang, Liu, & Wang, 2023) based on neural networks (Chandrasekar, Radhika, & Zhu, 2022a, 2022b; Chandrasekar, Rakkiyappan, & Li, 2016; Radhika, Chandrasekar, Vijayakumar, & Zhu, 2023) simulates these activities by manipulating the robot to reach certain desired goals (Huang et al., 2021; Rao et al., 2021; Santucci, Baldassarre, & Mirolli, 2016; Tamil Thendral, Ganesh Babu, Chandrasekar, & Cao, 2022; Zhang, Wang, & Gao, 2021). GCRL can be optimized as conventional reinforcement learning (RL) for robotic tasks (Bramlage & Cortese, 2022; Fu et al., 2022; Li, Luo, Song, & Yang, 2023; Ming, Gao, Liu, & Zhao, 2023; Tsantekidis, Passalis, & Tefas, 2023) using value or policy-based RL methods.

GCRL encounters challenges in diverse desired goals (Fang, Zhou, Du, Han, & Zhang, 2019), handling sparse rewards (Andrychowicz et al., 2017), and designing effective exploration (Kumar et al., 2021) strategies in complex environments, impeding the learning of versatile and robust policies. However, the issue of sparse reward poses a major

challenge in GCRL, where non-sparse reward signals are received when the desired goals are reached. The sparse reward in GCRL lies in the limited feedback signals, making it difficult for the policy to discern the impact of performed actions when reaching the desired goals. This issue hampers the training of GCRL policies as the lack of frequent and meaningful rewards leads to exploration difficulties. This issue can be alleviated by sampling the temporary goals to reach in the short term and gradually move toward the difficult desired goal. The temporary goals can be regarded as subgoals, which can be reached in the short term to design non-sparse reward signals for effective policy training.

The sampling of appropriate subgoals plays a crucial role in addressing the problem of sparse rewards (Andrychowicz et al., 2017; Bing et al., 2021, 2022). The subgoals can be considered as related desired goals, facilitating the design of training samples with non-sparse rewards. Hindsight experience replay (HER) (Andrychowicz et al., 2017; Bai et al., 2023; Vecchietti, Seo, & Har, 2022) considers the achieved goals in visited trajectories as subgoals for learning goal-conditioned action values. HER (Andrychowicz et al., 2017) aims to generalize the learned action values based on the sampled subgoals to desired goal-conditioned action value learning. However, it is essential

* Corresponding author.

E-mail addresses: yao.l@sxu.edu.cn (Y. Li), yuhui.wang@kaust.edu.sa (Y. Wang), x.tan@nuaa.edu.cn (X. Tan).

<https://doi.org/10.1016/j.neunet.2024.106825>

Received 7 July 2023; Received in revised form 8 August 2024; Accepted 14 October 2024

Available online 28 October 2024

0893-6080/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

to note that the uniformly sampled subgoals in HER (Andrychowicz et al., 2017; Fang et al., 2019) may not adequately represent or be typical of the critical positions required to reach the desired goal. Randomly sampled subgoals in HER (Fang et al., 2019) can negatively impact goal-conditioned action value learning efficiency. In addition, some goal-conditioned reinforcement learning methods (Nair et al., 2018; Nasiriany, Pong, Lin, & Levine, 2019b) generate subgoals by maximizing the goal-conditioned action value based on the generated goals to alleviate sparse rewards. Although the subgoals derived from the maximized action value allow for effective goal-conditioned action value learning, they may be overly simplistic, increasing the number of meaningless steps when reaching the desired goal. Overall, current subgoal generation methods (Nair et al., 2018; Nasiriany et al., 2019b; Pong et al., 2020) often overlook the context of specific tasks and the need for an appropriate distance between states and subgoals. To address this issue, it is beneficial to sample highly valued subgoals for effective goal-conditioned policy learning.

Sampling highly valued subgoals facilitates efficient goal-conditioned action value learning (Nachum, Gu, Lee, & Levine, 2018; Nair et al., 2018). Highly valued subgoals should meet two essential characteristics for efficient goal-conditioned action value learning. Firstly, the subgoals should be conditioned on the specific task contexts, which are captured by the context variable representing the latent representation of visited trajectories. This learned latent representation empowers the generator to sample subgoals gradually, reaching the desired goal. Secondly, subgoals should possess appropriate levels of difficulty at different training stages. These difficulty levels can be estimated using goal-conditioned action values (Chane-Sane, Schmid, & Laptev, 2021; Nasiriany et al., 2019b), which can be constrained within self-adaptive regularized ranges for a better tradeoff between exploration and exploitation. The subgoals, enabling the gradual reaching of the desired goal while maintaining a balance between exploration and exploitation, determine effective goal-conditioned policy learning.

In this paper, we propose a method for generating highly valued subgoals to facilitate efficient goal-conditioned action value learning. The subgoal generator is trained based on the visited trajectories by conditional VAE (Sohn, Lee, & Yan, 2015). The encoder model captures the context of the tasks represented by a Gaussian distribution based on past achievements, while the decoder model (Nair et al., 2018; Nasiriany et al., 2019b) takes the states and the desired goal conditioned on the task contexts to generate subgoals. Furthermore, the generated subgoals are appropriately constrained in terms of suitable complexity based on regularized goal-conditioned action values (Li et al., 2022; Zhang, Gan, & Tan, 2022). The action values based on the future generated subgoals are regularized by the action values based on the achieved goals to protect the derived subgoals from being excessively easy or difficult. The highlights of our subgoals are generated based on the task context and characterized by appropriate complexity. The task context gradually guides the agent toward the desired goal, while the suitable complexity balances exploration and exploitation, facilitating practical goal-conditioned policy training. The contributions of this work are summarized as follows:

- We propose a subgoal generation module based on the context of specific tasks, allowing for more effective planning to accomplish the tasks.
- We design a self-adaptive action value regularization mechanism to enable the generated subgoals to maintain suitable distances from the states.

We run our method Highly-Valued Subgoal Generation (HVSG) on the well-studied deep deterministic policy gradient (Fujimoto, van Hoof, & Meger, 2018; Lillicrap et al., 2016) reinforcement learning method in robotic environments. This paper first introduces the related works of goal-conditioned reinforcement learning in Section 2. The preliminaries of goal-conditional reinforcement learning were introduced in Section 3, and the issue of HER is raised in Section 3. Then, the proposed method is presented in Section 4, and the experiments are also reported in Section 5. Finally, we make conclusions about our method in Section 6.

2. Related works

GCRL (Andrychowicz et al., 2017; Liu, Trott, Socher, & Xiong, 2019) aims to accomplish a specific task. The related works of hindsight experience replay (Andrychowicz et al., 2017; Fang et al., 2019), hierarchical reinforcement learning (Nachum et al., 2018; Alex et al., 2017; Zhang, Guo, Tan, Hu, & Chen, 2020), and subgoals sampling (Chane-Sane et al., 2021) are introduced in this section.

2.1. Hindsight experience replay

Hindsight experience replay (Andrychowicz et al., 2017; Bai, Wang et al., 2023; Bing et al., 2022; Vecchietti et al., 2022) addresses the issue of sparse rewards in GCRL by treating the finally achieved goal as the desired goal to learn the goal-conditioned action value. Specifically, the visited trajectories are used to design easily achieved subgoals to reach, where some of the no sparse reward signals are designed, enabling efficient goal-conditioned policy training. HER (Andrychowicz et al., 2017) first achieves the most excellent performance in goal-conditioned reinforcement learning. However, only some of the sampled goals in the visited trajectories help guide the agent to reach the desired goal. Curriculum-guided HER (Fang et al., 2019) (CHER) selects the failed experiences for replay in the criteria that ensures curiosity in the earlier stage and improves the proportion of the goal proximity. However, the subgoals used in HER and CHER are sampled heuristically from the achieved goals. The sampled subgoals involving the context of the tasks and moderate difficulty may enhance the efficiency of goal-conditioned policy learning.

Reinforcement imagined goals (RIG) (Nair et al., 2018) samples the subgoals from the latent distribution learned by VAE to deploy HER in the visual task. The reward function is built on the distances between the states and the subgoals in the latent space. However, the subgoals are sampled from the goal space randomly. Therefore, some of the generated subgoals may limit the learning speed. Skew-Fit (Pong et al., 2020) maximizing the entropy of the generated goals learned based on RIG to improve the diversity of the sampled subgoals. Compared with our method, RIG (Nair et al., 2018) and Skew-Fit (Pong et al., 2020) generate the subgoals from the latent distribution without enforcing the generated subgoals characterized with high relation to the policy for easy reach. The sampled subgoals in RIG (Nair et al., 2018) and Skew-Fit (Pong et al., 2020) show a similar challenge in HER. This issue can be alleviated by sampling the subgoals conditioned on the specific tasks and keeping appropriate difficult level. These methods generate suitable subgoals by maximizing goal-conditioned action values. Nevertheless, subgoals derived from maximized action values may be too straightforward, increasing meaningless steps when attempting to reach the desired goal. The module for regularized action value derived subgoal generation stands out as the most distinctive among these methods, emphasizing the most challenging aspect of generating highly valued subgoals.

2.2. Subgoals generation

Subgoal generating also plays an essential role in goal-conditioned reinforcement learning, in which the generated subgoals are characterized by being easy to achieve, improving learning efficiency. Hindsight Planner (Lai, Wang, Yang, Zhu, & Kuang, 2020) trains the subgoals generative learner based on the past visited trajectories implemented by LSTM (Rakkiyappan, Chandrasekar, & Cao, 2014). RIS (Chane-Sane et al., 2021) samples the subgoals that are easier than the desired goal, in which the imaged goals are the midpoints from the current states to the desired goal. The goal-conditioned action value is used to quantify the distances between the states and goals, and the action values between the states and imaged goals are maximized to train the goal-generating learner. Goal GAN (Florensa, Held, Geng, & Abbeel, 2018) generates the subgoals based on the generative adversarial network, in

which the goals that the agent intends to reach and does not are labeled by the probability of reaching the goal over the limited time steps. The candidate subgoals should stimulate the agent to receive useful rewards. Our method is similar to Hindsight Planner (Lai et al., 2020) in self-learning, but the context of the tasks and the regularized action values (Li, Wang, Gan, & Tan, 2022; Zhang et al., 2022) are not used to derive reasonable goals. Compared with RIS (Chane-Sane et al., 2021), our method uses the regularized action value to protect the subgoals from too hard to efficient policy training.

Hierarchical reinforcement learning (Nachum et al., 2018; Alex et al., 2017; Zhang et al., 2020) also incorporates the concepts of subgoal sampling and reaching. Hierarchical goal-conditioned Reinforcement Learning (HRL) aims to guide the agent to reach the desired goal by encouraging the agent to reach the subgoals step one step. FeUdal Network (Alex et al., 2017) enforces the agent to achieve long-term goals through hierarchical subgoals represented by the latent variable. Hierarchical Reinforcement Learning with Off-policy Correction (HIRO) (Nachum et al., 2018) proposed an off-policy corrections technique for higher-level policy training to enable deploying HRL methods in real-world problems, allowing the agent to learn more efficiently and robustly. Hierarchical Adjacency Constrained Learning (HACL) (Zhang et al., 2020) proposed an adjacency constraint mechanism to generate suitable subgoals by shortest transition distance measurement for efficient HRL policy training. The hierarchical reinforcement learning (HRL) (Nachum et al., 2018) method shares similarities with HER (Andrychowicz et al., 2017). However, the key distinction lies in whether the subgoals are explicitly enforced to be reached. In the case of HER (Andrychowicz et al., 2017), the agent is encouraged to implicitly reach the subgoal by leveraging the generalized goal-conditioned action values. In contrast, in hierarchical reinforcement learning (HRL), the agent is explicitly compelled to achieve the subgoals leading to the desired goal gradually.

3. Preliminaries

Goal-conditioned Reinforcement Learning Goal-conditioned reinforcement learning aims to manipulate the robot to reach a certain desired goal. An infinite horizon γ -discounted MDP in GCRL consists of six elements denoted as $\langle S, \mathcal{G}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma \rangle$, where S denotes state space, $\mathcal{G}$ represents the goal space, $\mathcal{A}$ represents action space and $\mathcal{P}$ indicates transition probability from state s_t to state s_{t+1} by performing action a_t , the *achieved goal* ag_t in the space of $\mathcal{G}$ is also received from the environment, and the achieved goals can be regarded as the mapping results of the states $ag_t = \varphi(s_t)$, g represents the *desired goal* that the agent want to achieved, $\mathcal{R}$ denotes reward signal $r(s_t, a_t, g)$ received from the environment, and the definition of the reward function is formulated as follows:

$$r(s_t, a_t, g) = \begin{cases} 0 & d(\varphi(s_t), g) < \delta \\ -1 & \text{otherwise} \end{cases} \quad (1)$$

where δ represents the threshold for the distance $d(\varphi(s_t), g)$ between the achieved goal $\varphi(s_t)$ and the desired goal g , γ represents the discounted factor controlling the importance of short-term rewards for the current state. The performed action used to interact with the environment is learned based on the current state s_t and the desired goal g as deterministic policy $a_t = \pi(s_t, g)$.

The objective of goal-conditioned RL is maximizing the expected return concerning the goal distribution: $J(\pi) = \mathbb{E}_{g \sim \rho_g, \tau \sim d^\pi(\cdot|g)} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, g)]$, where $d^\pi(\tau|g) = \rho_0(s_0) \prod_t \pi(a_t|s_t, g) p(s_{t+1}|s_t, a_t)$, $\rho_0(s_0)$ represents the initial state distribution, $\pi(a_t|s_t, g)$ means the goal-conditioned policy, $p(s_{t+1}|s_t, a_t)$ represents the transition probability. Goal-conditioned reinforcement learning can be solved by learning the goal-conditioned action value function $Q^\pi(s, g, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, g) | s_0 = s, a_0 = a]$, which represents the expected return starting from state s and action a to accomplish the desired goal g .

Goal-conditioned reinforcement learning poses a significant challenge due to sparse rewards (Andrychowicz et al., 2017; Devidze, Kamalaruban, & Singla, 2022; Sohn et al., 2021). Meaningful rewards are only obtained from the environment upon successfully reaching the desired goal with a value of -1 , while other states correspond to -1 . In particular, the reward sequence is $(\underbrace{-1, \dots, -1}_{T-1}, \underbrace{0}_T)$, featuring

mostly -1 rewards with a single zero reward at time step T . This scenario hinders the policy from distinguishing the advantages of the action sequence, adversely affecting overall policy performance.

Deep Deterministic Policy Gradient Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2016) is a fundamental reinforcement learning method comprising an Actor and Critic module. The Actor interacts with the environment and trains the policy, while the Critic module evaluates the policy. Many contemporary reinforcement learning methods (Andrychowicz et al., 2017; Fujimoto et al., 2018) for continuous tasks are built upon the DDPG RL method, incorporating modifications to actor training or critic update strategies for more efficient policy learning. Our method, based on the DDPG framework, adapts the construction of training samples to tackle the challenge of sparse rewards. Additionally, the update procedures for both the Actor and Critic remain consistent with DDPG.

Hindsight Experience Replay HER (Andrychowicz et al., 2017) alleviates the issue of sparse reward by replacing some of the desired goals g in the training samples with the achieved goals $\hat{g}'_t = \varphi(s_{t+k})$, $k \geq 1$ in the visited trajectories, and the corresponding rewards are also changed. Given the desired goal g and the visited trajectory $\tau = \{(s_t, g, a_t)\}_{t=0}^{T-1}$, DDPG (Lillicrap et al., 2016) trains the action values based on the tuples $(s_t \| g, a_t, s_{t+1} \| g, r(s_t, a_t, g))$, in which $r(s_t, a_t, g) = 0$ only when the distances between the achieved goals $\varphi(s_t)$ and the desired goal g are no more than the threshold δ . HER (Andrychowicz et al., 2017) trains the action values based on the tuples $(s_t \| g, a_t, s_{t+1} \| g, r(s_t, a_t, g))$ and the designed tuples $(s_t \| \hat{g}'_t, a_t, s_{t+1} \| \hat{g}'_t, r(s_t, a_t, \hat{g}'_t))$, in which the sampled subgoals $\hat{g}'_t$ are the achieved goals in the visited trajectories. The sampled subgoals $\hat{g}'_t$ are the positions that are closing to the states s_t , such that most of the reward signals in HER are zero as $r(s_t, a_t, \hat{g}'_t) = 0$ with useful rewards.

4. Method

Goal-conditioned reinforcement learning suffers from the issue of sparse rewards, which impacts the learning efficiency of goal-conditioned action value and adversely affects the learned goal-conditioned policy. Our method addresses this challenge by focusing on generating highly valued subgoals to alleviate sparse rewards. The subgoals are generated based on the context of the tasks and characterized by suitable complexity, allowing for the generation of more appropriate subgoals for policy training.

4.1. Framework

Fig. 1 illustrates the diagram of our proposed method, where the behavior policy $\pi^b(s, g)$ interacts with the environment, collecting the visited samples $(s_{\tau 1, t} \| g, a_{\tau 1, t}, s_{\tau 1, t+1} \| g, r_{\tau 1, t}), (s_{\tau 2, t} \| g, a_{\tau 2, t}, s_{\tau 2, t+1} \| g, r_{\tau 2, t})$ in two different trajectories. These visited samples are utilized to train the subgoal generation learner $\hat{g}'_t = f_\phi(s_t, \hat{g})$ in the first step. Subsequently, the subgoal is generated by replacing the achieved goal $\hat{g}$ with the desired goal $g'_t = f_\phi(s_t, g)$ in the second step. Finally, the regularized goal-conditioned action values are employed for recodifying samples to generate highly valued subgoals and then utilized for efficient goal-conditioned action value training and policy learning. Overall, our method consists of two submodules for highly valued subgoal generation: one is the context-based subgoal generation module, and the other is the highly valued subgoal regularization module.

Fig. 2 illustrates the working mechanism of our proposed subgoal generator. For example, when given the desired goal g and the visited

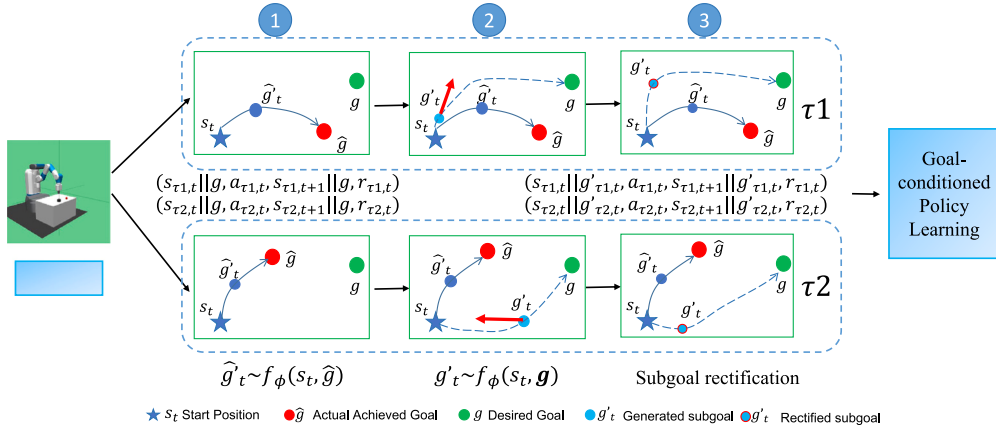


Fig. 1. The framework of our proposed highly valued subgoal generation.

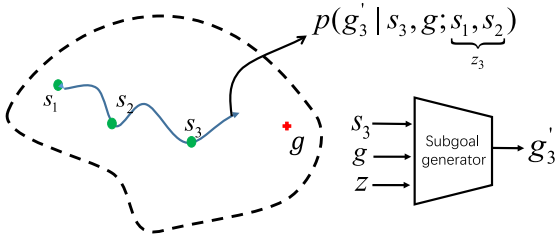


Fig. 2. The idea of subgoal generation. The dotted line represents the goal space, and the subgoal g'_3 is sampled from the conditional distribution based on the desired goal g and context task z_3 , which is learned based on the past achievements.

trajectories (s_1, s_2, s_3) , the subgoal g'_3 can be sampled from the distribution $g'_3 \sim p(g'_3 | s_3, g; s_1, s_2)$. The generation of subgoal g'_3 is conditioned on the state s_3 and the desired goal g , as well as the context variable z_3 , which represents the latent representation of past achievements (s_1, s_2) . The latent representation of (s_1, s_2) captures the context of the specific tasks, allowing for more suitable subgoal generation. For instance, in the context of a robot sorting items within a warehouse, the task-specific derived subgoals incorporate information such as the sequence of processed items, the current position of the robot, and its orientation. This mechanism based on the context tasks allows the generated subgoals to gradually reach the desired goal.

4.2. Context based subgoal generation

The context variable, encapsulated by the latent representation of past achievements, captures informative knowledge relevant to the environment. This enables the generated subgoals to guide the agent toward reaching the desired goal gradually. The visited trajectory τ can be used for learning the context variable based on the latent model with conditional VAE (Nasiriany et al., 2019b; Sohn et al., 2015) through variational inference. We can sample the subgoal g'_t from the conditional distribution as $g'_t \sim p(g'_t | s_t, \hat{g}, z_t)$, z_t represents the contextual information derived from the visited state sequence. The subgoal g'_t necessary to achieve a desired goal g can be sampled from a conditional distribution $g'_t \sim p(g'_t | s_t, g, z_t)$. Specifically, the latent representation z_t can be derived based on the encoder module, and the distribution $p(g'_t | s_t, \hat{g}, z_t)$ can be learned based on the decoder module in conditional VAE.

Fig. 4 shows the framework of our subgoal generator based on the visited trajectories. The visited trajectory $\tau = \{(s_i, g, a_i)\}_{i=0}^T$ ending with $\hat{g}$ enables learning the subgoal generator by learning how to map the state s_t and the achieved goal $\hat{g}$ to the sampled subgoal g'_t . Then the achieved goal $\hat{g}$ is replaced by the desired goal g to sample the subgoal

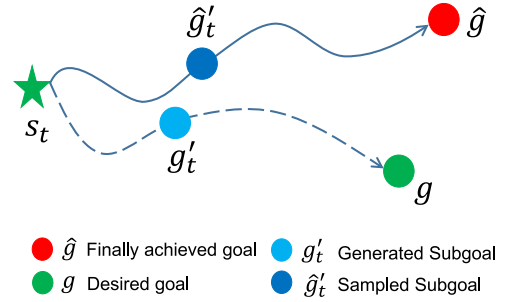


Fig. 3. The visualization of visited trajectories. The visited trajectory can be used to learn the mapping from s_t and $\hat{g}$ to the subgoal g'_t . And the subgoal g'_t for reaching the desired goal g can be generated based on the learned generator.

g'_t , as the solid line in Fig. 3. The subgoal generator can be implemented by the decoder model in conditional VAE (CVAE) (Sohn et al., 2015). The encoder and decoder are implemented by the Long Short-Term Memory (LSTM) module to capture the dependence on the timing sequence. Given the trajectory $\tau = \{(s_i, \hat{g}, a_i, g'_i)\}_{i=1}^T$, define $\hat{g} = \varphi(s_T)$, $g'_i = \varphi(s_{i+k})$, $k > 1$, the input of the encoder in each cell is $(s_i, \hat{g}, g'_i)$, and the output of the encoder is $h_i^e = \text{Enc}_\psi(s_i, \hat{g}, g'_i)$, which is followed by a full connection layer to learn a Gaussian distribution $\mathcal{N}(\mu_i, \sigma_i^2)$ with its mean μ_i and variance σ_i^2 . The input of the decoder in each cell is $(s_i, \hat{g}, z_i)$, in which the latent variable $z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ is sampled from the learned Gaussian distribution. The output of the decoder in each cell is the reconstructed subgoal $\hat{g}'_{d,i} = \text{Dec}_\phi(s_i, \hat{g} | z_i)$ given the state s_i , finally achieved goals $\hat{g}$, as well as the learned latent variable z_i . The objective function minimizes the distance between $\hat{g}'_{d,i}$ and g'_i as well as the KL divergence between the assumed prior distribution $\mathcal{N}(0, 1)$ and the learned distribution $\mathcal{N}(\mu_i, \sigma_i^2)$. The objective for the sequence $\hat{g}'_{0:t}$ and the reconstructed subgoal sequence $\hat{g}'_{d,0:t}$ formulates as minimizing Eq. (2).

$$\mathcal{L}_{\psi, \phi}(s_{0:t}, \hat{g}, g'_t) = \sum_i (\hat{g}'_{d,i} - g'_i)^2 + D_{KL}(\mathcal{N}(\mu_i, \sigma_i^2), \mathcal{N}(0, 1)) \quad (2)$$

where ψ and ϕ represent the parameters of the encoder and decoder modules, D_{KL} represents the KL divergence. The conditional VAE module learns the conditional distribution $p(g'_t | s_t, \hat{g}, z_t)$ of the subgoal for state s_t and the finally achieved goal $\hat{g}$ in the visited trajectories. In practice, the input and output in an LSTM (Long Short-Term Memory) are sequences. The decoder model generates the subgoal sequence $g'_{0:t}$ based on the state sequence $s_{0:t}$, and we randomly select state s_t and subgoal g'_t pairs from the state and subgoal sequence for goal-conditioned action value learning.

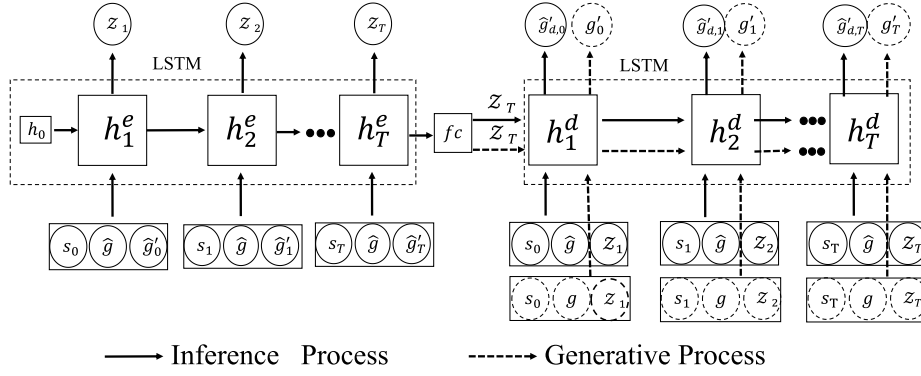


Fig. 4. The framework of our subgoal generator. The encoder and decoder are two LSTM modules. h_t^e represents the hidden layer in the encoder, and h_t^d represents the hidden layer in the decoder module. The dotted line circle represents the desired goal g , and $\hat{g}$ represents the finally achieved goal.

4.3. Highly valued subgoal regularization

The suitable distances between the states and the subgoals constitute reasonable complexity for highly valued subgoal generation. The smaller distance between the state and subgoal is associated with a meaningful reward of 0, which increases the number of meaningless steps when reaching the desired goal. Conversely, a larger distance can help mitigate this issue, but it is linked to the issue of sparse rewards, with most rewards -1 . The goal-conditioned action values based on the generated subgoals can be used to estimate the difficulty level of the subgoals for specific states. The larger action value implies that a higher percentage of the rewards is zero, corresponding to being easy to reach. Conversely, the smaller action value implies that a higher percentage of the rewards is -1 , corresponding to being hard to reach. The suitable subgoals can be derived by maximizing or minimizing the goal-conditioned action values to control the distances between the states and the generated subgoals. Specifically, the generated subgoal g'_t derived by maximizing its goal-conditioned action value $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ enables the subgoal to be easy to reach for exploitation, while minimizing its goal-conditioned action value $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ enables the subgoal to be hard to reach for exploration. However, the maximized value-derived subgoals may be too easy to increase the meaningful steps, and the minimized value-derived subgoals may be too hard to alleviate the sparse rewards.

To address the issue of subgoals being excessively challenging or simplistic, it is necessary to impose constraints on the action values within specific ranges. If a generated subgoal g'_t is nearby or too distant from the state s_t , it can be pushed or pulled toward the state with similar distance between the achieved goal $\hat{g}'_t$ and state s_t . The achieved goal $\hat{g}'_t$ can be used to establish upper and lower bounds for the maximized and minimized values $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ of the derived subgoal g'_t . The goal-conditioned action values $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ representing states and goals distances show radical values compared with $Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t))$, the action values $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ will be regularized to constrain the generated subgoals, ensuring suitable distances with the current states. Specifically, the action value $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ regularized by $Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t))$ can be maximized or minimized in different settings to enforce the generated subgoal characterized by the suitable complexity.

Fig. 5 illustrates the subgoal generation guided by regularized action values, where a higher action value corresponds to a smaller distance between state and goal, and vice versa. In the top, $Q_\theta(s_1, g'_1, \pi(s_1, g'_1)) \leq Q_\theta(s_1, \hat{g}'_1, \pi(s_1, \hat{g}'_1))$ suggests that the generated goal g'_1 is farther away from the current state compared to the future achieved goal $\hat{g}'_1$. To mitigate this issue, the generated subgoal g'_1 is pulled toward the state to reduce its distance. In the bottom, $Q_\theta(s_2, g'_2, \pi(s_2, g'_2)) > Q_\theta(s_2, \hat{g}'_2, \pi(s_2, \hat{g}'_2))$ suggests that the generated goal g'_2 is too close to the current state compared to the future achieved goal $\hat{g}'_2$. To alleviate this issue, the

generated subgoal g'_2 is pushed toward the state to increase its distance. The objective for subgoals generating is minimizing the objective:

$$\mathcal{L}_{\psi, \phi}^{\text{HVSg}}(s_{0:t}, \hat{g}, \hat{g}') = \mathcal{L}_{\psi, \phi}(s_{0:t}, \hat{g}, \hat{g}') - \alpha \mathcal{L}_{\phi}^{\text{hv}}(s_t, g, \hat{g}') \quad (3)$$

where $\mathcal{L}_{\phi}^{\text{hv}}(s_t, g, \hat{g}')$ represents the regularization term for generating the highly valued subgoals, coefficient $\alpha > 0$ is the parameter of the regularization term. Notably, our conditional VAE is trained based on the visited trajectories, and the regularization term is trained based on the data sampled from the state sequence $s_{0:t}$ and the corresponding generated subgoal sequences $g'_{0:t}$. The regularization term is formulated by:

$$\mathcal{L}_{\phi}^{\text{hv}}(s_t, g, \hat{g}') = \mathbb{E}_{z_t \sim \mathcal{N}(\mu_t, \sigma_t^2)} \mathbb{I}(Q_{s_t, g'_t, \hat{g}'_t}) Q_\theta(s_t, g'_t, \pi(s_t, g'_t)) \quad (4)$$

$$\mathbb{I}(Q_{s_t, g'_t, \hat{g}'_t}) = \begin{cases} 1 & Q_\theta(s_t, g'_t, \pi(s_t, g'_t)) \leq Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t)) \quad (a) \\ -1 & Q_\theta(s_t, g'_t, \pi(s_t, g'_t)) > Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t)) \quad (b) \end{cases}$$

where the subgoal g'_t for state s_t can be generated by $g'_t = \text{Dec}_\phi(s_t, g|z_t)$. $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ is the action value conditioned on the state s_t and the generated subgoal g'_t . $Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t))$ is the action value conditioned on the state s_t and the goal $\hat{g}'_t$ that the agent has been reached sampled as HER (Andrychowicz et al., 2017). The operator $\mathbb{I}(Q_{s_t, g'_t, \hat{g}'_t})$ is an indicator function, $\mathbb{I}(Q_{s_t, g'_t, \hat{g}'_t}) = 1$ if $Q_\theta(s_t, g'_t, \pi(s_t, g'_t)) \leq Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t))$, and $\mathbb{I}(Q_{s_t, g'_t, \hat{g}'_t}) = -1$ if $Q_\theta(s_t, g'_t, \pi(s_t, g'_t)) > Q_\theta(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t))$. The different conditions correspond to various settings for subgoal generation: For condition (a), this condition indicates that the generated subgoals g'_t may be too far away from the states, as shown by the top line in Fig. 5. The regularization term in Eq. (4) alleviates this issue by maximizing the action value $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ to reduce their distances, thus preventing the generated subgoals from straying too far from the states. For condition (b), this condition indicates that the generated subgoals g'_t may be too close to the states, as depicted by the bottom line in Fig. 5. The regularization term in Eq. (4) addresses this issue by minimizing the action value $Q_\theta(s_t, g'_t, \pi(s_t, g'_t))$ to increase their distances, protecting the generated subgoals from being too close to the states. Ideally, the generated subgoals g'_t are pushed or pulled toward states that result in similar distances between s_t and g'_t , compared with the distances between the states s_t and achieved goals $\hat{g}'_t$, through iterative training under conditions (a) and (b). Even the generated subgoals g'_t and the achieved subgoals $\hat{g}'_t$ exhibit similar distances from states s_t , g'_t facilitates guiding the agent to reach the desired goal, whereas $\hat{g}'_t$ does not.

Eq. (3) gives the objective for subgoal generator training based on the visited trajectories, and Eq. (4) is the regularization term for sampling the suitable subgoals for efficient policy training. The trigger conditions in Eq. (4) correspond to distinct subgoals for exploration

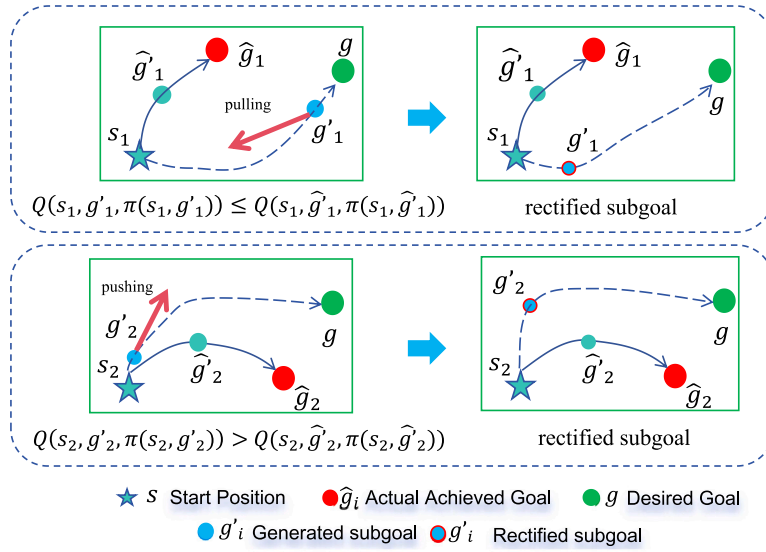


Fig. 5. The illustration of the regularized goal-conditioned action value for subgoal generation.

and exploitation. The subgoals in close proximity to states, obtained by maximizing the action values, facilitate the exploitation of the environment. Conversely, the subgoals distant from states by minimizing the action values promote environmental exploration. The subgoal generation for exploration and exploitation is alternated adaptively to facilitate the efficiency of goal-conditioned policy learning.

4.4. Discussion

Our method involves training the subgoal generator by leveraging information from visited trajectories. The diversity of the states in these trajectories plays a crucial role in determining the generalization capability of the learned subgoal generator. This underscores the dependence of our method on the exploration conducted by the goal-conditioned policy. It is essential to note that our method may face challenges when applied to more complex goal-conditioned tasks, such as those posed by Rubik's Cube, due to the limitations imposed by this dependence on exploration.

HER (Andrychowicz et al., 2017) uniformly samples subgoals from achieved goals in visited trajectories to design non-sparse training samples for action value learning. Our method generates highly valued subgoals conditioned on the context information of the tasks for action value learning, and the generated goals are also keeping adaptive distance with the current state in different training stages. Algorithm 1 shows the diagram of our proposed method, where the difference from HER is the bold subgoal sampling mechanism. Additionally, our method belongs to the category of serializable subgoal generation methods, in contrast to HER (Andrychowicz et al., 2017), which is a non-serializable goal generation method.

Some variants of our method are also introduced for easy discussion in experiments. **Goal-conditioned Subgoal Generation (GCSG)** only trains the subgoal generator based on the self-designed demonstrations with $\alpha = 0$. **GCSGMax** generates the subgoal by maximizing the action value without regularized value, and the objective of GCSGMax is formulated as minimizing:

$$\mathcal{L}_{\psi, \phi}(s_t, \hat{g}, \hat{g}'_t) - \alpha Q_{\theta}(s_t, g'_t, \pi(s_t, g'_t)) \quad (5)$$

and the objective of **GCSGMaxClip** is minimizing

$$\mathcal{L}_{\psi, \phi}(s_t, \hat{g}, \hat{g}'_t) - \alpha [Q_{\theta}(s_t, g'_t, \pi(s_t, g'_t))]_{Q_{\theta}(s_t, g'_t, \pi(s_t, g'_t)) \leq Q_{\theta}(s_t, \hat{g}'_t, \pi(s_t, \hat{g}'_t))} \quad (6)$$

Algorithm 1 Highly Valued Subgoal Generation

- 1: Initialize actor network ω and critic network θ with random weights
- 2: Initialize target networks ω' and θ' with the weights as the actor and critic: $\omega' \leftarrow \omega, \theta' \leftarrow \theta$
- 3: Initialize replay buffer R and a conditional VAE $G = \{Enc_{\psi}, Dec_{\phi}\}$
- 4: **for** episode=1,M **do**
- 5: Sample an initial state s_0 and a desired goal g
- 6: **for** t=0,T-1 **do**
- 7: Sample an action a_t on the behavior policy of DDPG $a_t \leftarrow \pi_b(s_t|g)$
- 8: Execute the action a_t , observe a new state s_{t+1}
- 9: **end for**
- 10: **for** t=0,T-1 **do**
- 11: Store the transition $r_t := r(s_t, a_t, g)$ in R
- 12: **Sample the subgoals g'_t as $g'_t = Dec_{\phi}(s_t, g, |z_t)$**
- 13: **Store tuples $(s_t, g'_t, a_t, r_t(s_t, a_t, g'_t), s_{t+1}, g'_t)$ in R**
- 14: **end for**
- 15: **for** t=1,J **do**
- 16: Sample transitions from R , and optimizing generator on Eqn. (3)
- 17: Compute target value: $y = r(s_t, g'_t, a_t) + \gamma \cdot Q(s_{t+1}, g'_t, \pi(s_{t+1}, g'_t)|\omega')|\theta')$
- 18: Update critic by minimizing: $\mathcal{L}(\theta) = \frac{1}{N} \sum [Q(s_t, g'_t, a_t|\theta) - y]^2$
- 19: Update actor network using the sampled policy gradient: $\nabla_{\omega} \pi \approx \frac{1}{N} \sum \nabla_a Q(s_t, g'_t, a|\phi)|_{s=s_t, a=\pi(s_t, g'_t|\omega)} \nabla_{\omega} \pi(s_t, g'_t|\omega)|_{s_t}$
- 20: Update target networks: $\omega' \leftarrow \beta \omega + (1 - \beta)\omega'; \theta' \leftarrow \beta \theta + (1 - \beta)\theta'$
- 21: **end for**
- 22: **end for**

where $[x]_{condition} = x$ if satisfying this *condition*, and $[x]_{condition} = 0$ if not satisfying this *condition*. **GCSGMin** generates the subgoal by minimizing the action value without using the clipped operator. The objective of GCSGMin is formulated as minimizing:

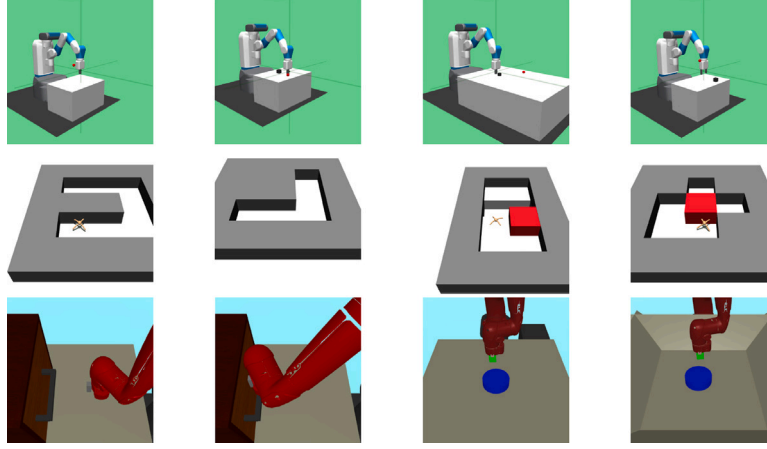


Fig. 6. The visualization on robotic goal-conditioned examples. The top is the Fetch based tasks, the middle is the Ant based, and the bottom is the Sawyer based tasks.

$$\mathcal{L}_{\psi, \phi}(s_t, \hat{g}_t, \hat{g}'_t) + \alpha Q_{\theta}(s_t, g'_t, \pi(s_t, g'_t)) \quad (7)$$

and the objective of **GCSGMinClip** is minimizing

$$\mathcal{L}_{\psi, \phi}(s_t, \hat{g}_t, \hat{g}'_t) + \alpha [Q_{\theta}(s_t, g'_t, \pi(s_t, g'_t)) - Q_{\theta}(s_t, g'_t, \pi(s_t, g'_t))] \quad (8)$$

Overall, Eqs. (5), (6), (7), and (8) represent variant methods compared to Eq. (3). Specifically, Eq. (5) and Eq. (6) provide the objectives for subgoal generation by maximizing the goal-conditioned action values. On the other hand, Eqs. (7) and (8) present the objectives for subgoal generation methods that focus on minimizing the action values. The related methods, GCSG, GCSGMax, GCSGMaxClip, GCSGMin, and GCSGMinClip, will be deliberately discussed in the experiments to analyze the contributions of our proposed modules for highly valued subgoal generation.

5. Experiments

5.1. Environments

We consider robotic manipulation tasks in goal-conditioned reinforcement learning environments to evaluate our proposed method. Besides, one navigation task, PointEnv, is used to investigate the generated subgoals in different settings. Some of the environments are illustrated in Fig. 6. These environments consist of the Fetch tasks, including Reaching, Pushing, Sliding, PickAndPlace, and the Ant-based tasks, involving AntMaze, AntMazeL, AntPush, AntFall, and the Sawyer tasks, including DoorHookResetFreeEnv, DoorOpen, PushAndReachEnvHard, PushAndReachArenaEnv, from left to right and top to bottom. The Sawyer based controlling tasks DoorHookResetFreeEnv, DoorOpen, PushAndReachEnvHard, PushAndReachArenaEnv are abbreviated by SawyerDHRF, SawyerDoorOpen, SawyerPAREH, SawyerPARAE, and the dimension of the state, goal and action are also reported in Table 1.

5.2. Implementation details

Our experiments are trained on a single machine with 1 InterXeon(R)W-2255 CPU and Geforce RTX 3090 GPU, and each task is stimulated with 20 roll-outs. Each agent is trained for 50 epochs with batch size 256, and the other hyper-parameters are the same as in Andrychowicz et al. (2017), except the training epoch of 20K for AntMaze and 10K for AntMazeL, AntPush, and AntFall. The subgoals used for action values learning consist of half of the subgoals sampled in HER and half of those generated by the conditional VAE for stable training as Nair et al. (2018). The sampled latent variable is clipped in $(-0.5, 0.5)$. The weight α of the regularization term in Fetch tasks is 0.01, and 1 in Sawyer tasks. Our method is built based on a combination of a subgoal generator, along with actor and critic modules

Table 1

The dimension of state, goal, action in Fetch, Sawyer and Ant tasks.

Environment	State	Goal	Action
Reaching	10	3	4
Pushing	25	3	4
Sliding	25	3	4
PickAndPlace	25	3	4
SawyerDoorHookResetFreeEnv	4	4	3
SawyerDoorOpen	4	4	3
SawyerPushAndReachArenaEnv	5	5	2
SawyerPushAndReachEnvHard	5	5	2
AntMaze	30	2	8
AntMazeL	30	2	8
AntPush	30	2	8
AntFall	30	2	8

implemented in DDPG (Lillicrap et al., 2016). The subgoal generator is realized as an encoder-decoder model featuring three layers built upon LSTM architecture. The actor and critic modules comprise three hidden layers each, utilizing a fully connected layer activated by ReLU with 256 neurons. The hidden layers in LSTM also consist of 256 neurons activated by Tanh. For both the subgoal generator and policy learning, we employ the Adam optimizer with a learning rate of $3e-4$. The parameter k used for sampling future achieved goals is a random variable within the range of $[1, T - t)$, where T denotes the maximum steps.

We compared our method **HVSG** with the following baselines: **DDPG** (Lillicrap et al., 2016), a well-known reinforcement learning method for continuous tasks; **HER** (Andrychowicz et al., 2017), a baseline method that uses hindsight experience replay to address sparse rewards; **TD3** (Fujimoto et al., 2018), a well-known RL method to alleviate the issue of overestimation; **CHER** (Fang et al., 2019), a curriculum-guided hindsight experience replay method that adaptively and dynamically controls the exploration-exploitation trade-off during training; **RIG** (Nair et al., 2018), a method where subgoals are sampled using the learned latent variable in VAE as representation learning; **WGCSL** (Yang et al., 2022), a method that learns the adaptive weight based on goal-conditioned supervised learning; **RIS** (Chane-Sane et al., 2021), a subgoal-generating method that maximizes the action values of the maximal values between the state subgoal tuples and the subgoal desired goal tuples; **LEAP** (Nasiriany et al., 2019b), a subgoal-generating method that maximizes the action values conditioned on the generated subgoals, where VAE is used to learn the latent representation of the goal samples; and **HIRO** (Nachum et al., 2018), a hierarchical reinforcement learning method with off-policy correction for stable goal-conditioned hierarchical policy training. We evaluate the HER-based methods HER (Andrychowicz et al., 2017), CHER (Fang et al.,

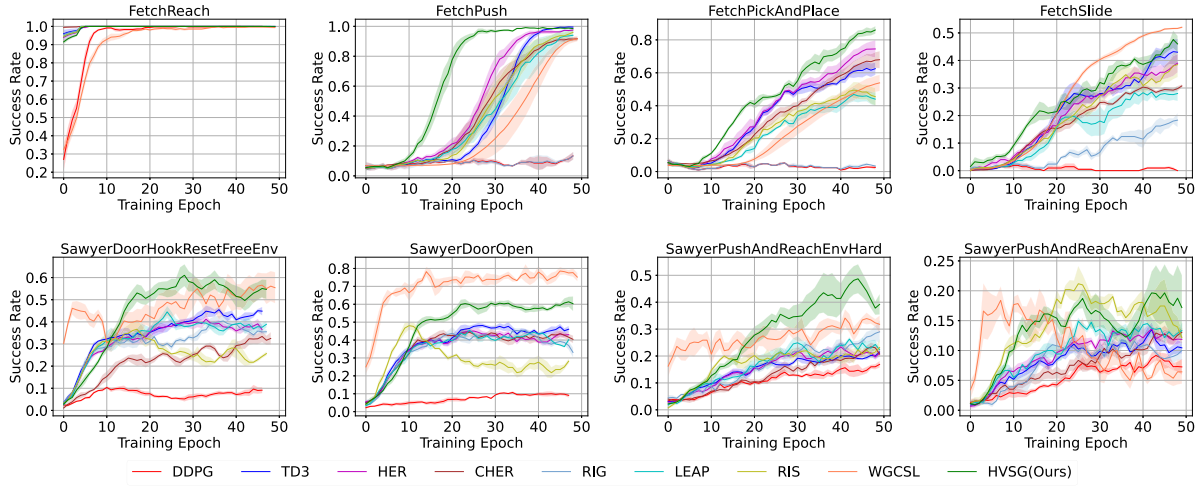


Fig. 7. Evaluation results in goal-conditioned tasks. The shaded region represents half a standard deviation of the average evaluation over five trials.

Table 2

The average evaluation performance in the term of success rate with five seeds, $\pm$ corresponds to a single standard deviation over trials.

Environment	DDPG	TD3	HER	CHER	RIG	LEAP	RIS	WGCSL	HVSG(Ours)
FetchReach	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
FetchPush	0.1333	0.9950	0.9750	0.9170	0.1333	0.9400	0.9600	0.9162	0.9850 $\pm$ 0.0002
FetchPickAndPlace	0.0250	0.6250	0.7450	0.6795	0.0333	0.4400	0.4550	0.5388	0.8600 $\pm$ 0.0024
FetchSlide	0.0	0.4300	0.3900	0.3075	0.1833	0.2800	0.3850	0.5204	0.4600 $\pm$ 0.0041
SawyerDHRF	0.0914	0.4483	0.3648	0.3250	0.3552	0.3889	0.2566	0.5543	0.5466 $\pm$ 0.0094
SawyerDoorOpen	0.0910	0.4605	0.4301	0.4042	0.3311	0.4017	0.2819	0.7518	0.6032 $\pm$ 0.0080
SawyerPAREH	0.1689	0.2151	0.2105	0.2068	0.2900	0.2235	0.2319	0.3179	0.3920 $\pm$ 0.0069
SawyerPARAE	0.0726	0.1044	0.1187	0.1306	0.1000	0.1333	0.1218	0.0643	0.1718 $\pm$ 0.0146

2019), RIS (Chane-Sane et al., 2021), LEAP (Nasiriany et al., 2019b), RIG (Nair et al., 2018) in the Fetch and Sawyer-based tasks, and evaluate the hierarchical reinforcement learning method HIRO (Nachum et al., 2018) in the Ant tasks.

5.3. Benchmark results

Fig. 7 reports the evaluation success rates on the benchmark environments, in which each compared method is evaluated ten times to report the average success rates. Evaluation results in Fig. 7 show that HER (Andrychowicz et al., 2017) using hindsight experience replay addresses the issue of sparse reward compared with DDPG, improving the performance by a large margin. We also compared our method HVSG with the well-known reinforcement learning algorithm TD3 (Fujimoto et al., 2018). The results indicate that TD3 (Fujimoto et al., 2018; Sivamayil et al., 2023) with HER (Andrychowicz et al., 2017) achieves a higher success rate compared to HER built based on DDPG (Lillicrap et al., 2016). This outcome stems from the fact that TD3 (Fujimoto et al., 2018) can be employed to address the issue of overestimation that DDPG often encounters. RIG (Nair et al., 2018) sampling the goals from the learned latent goal distribution shows bad performance because the sampled subgoals in RIG are generated randomly rather than conditioned on the desired goal. CHER (Fang et al., 2019) sampling the goals from the replay buffer is inferior to HER in some environments because the subgoals are sampled without taking the context of the tasks and the desired goal as a condition. RIS (Chane-Sane et al., 2021) and LEAP (Nasiriany et al., 2019b) maximizing the action values without constraint may sample the too-easy subgoals, increasing the meaningless steps. Our method, HVSG, is superior to most methods in terms of success rates and sample efficiency, which results from the subgoals conditioned on the task context and maintained adaptive distance with the state enabling efficient policy learning. Additionally, our method was compared with the well-known WGCSL (Yang et al., 2022). The results demonstrate that our method consistently exhibits robust

Table 3

The training time (Minutes) of one epoch in the Fetch tasks.

Environment	DDPG	HER	RIG	LEAP	RIS	HVSG
FetchPush	2.11 m	2.05 m	3.77 m	4.23 m	4.58 m	3.34 m
FetchPickAndPlace	2.01 m	1.99 m	3.66 m	4.15 m	4.47 m	3.27 m
FetchSlide	2.23 m	2.19 m	3.90 m	4.36 m	4.70 m	3.41 m
FetchReach	0.18 m	0.19 m	0.52 m	0.61 m	0.68 m	0.44 m

performance across various tasks compared to WGCSL. While WGCSL occasionally achieves high scores in FetchSlide and SawyerDoorOpen, it tends to have a lower success rate in other tasks as FetchPush and FetchPickAndPlace. Evaluation results in Fig. 7 demonstrate the effectiveness of our proposed subgoals sampling mechanism. The average success rates in the last three steps were reported in Table 2, and the variances across five results in our method were also reported with $\pm$ symbol. Results show that our method achieves the most stable performance compared with the baselines.

Our method involves subgoal generation and evaluation, leading to an increased training time for policy learning. Table 3 reports the training time based on one seed with one epoch in the Fetch tasks. Specifically, the training epoch of HVSG indicates that our method requires more training time in minutes compared to the baseline HER (Andrychowicz et al., 2017). This phenomenon results from the subgoal generation and evaluation module for efficient goal-conditioned action value learning. Additionally, compared with other subgoal generation methods, our approach achieves comparable training times because these methods involve subgoal generation and evaluation of action value.

5.4. Action value map analysis

Our method is trained based on the value-based reinforcement learning method DDPG (Lillicrap et al., 2016). Fig. 8 presents the action

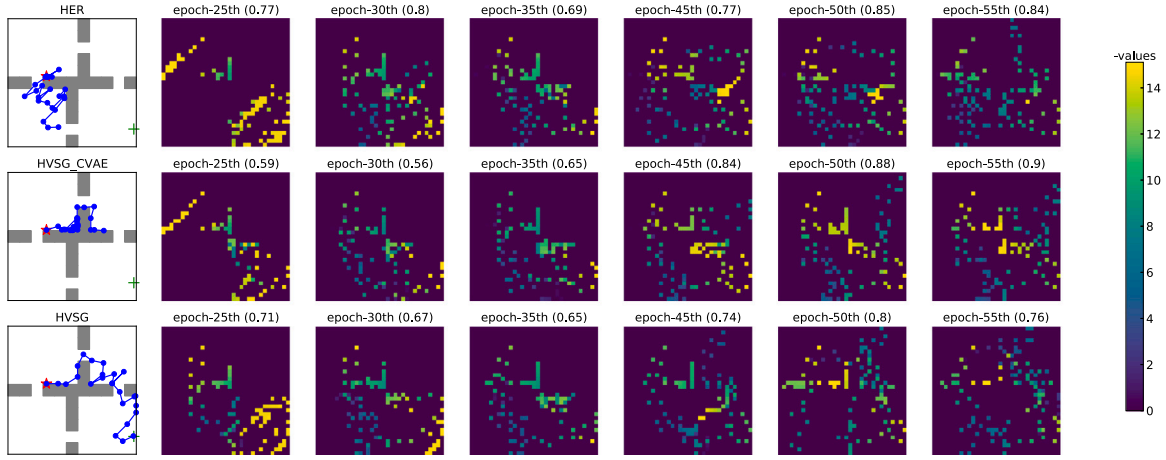


Fig. 8. The visualization shows the negative action values based on the states and the generated subgoals in the visited trajectories. The mean negative action values are reported in the brackets for clear visualization.

value maps of the navigation task in HER and our method, allowing us to investigate the effectiveness of our subgoal-generating method. We first sample 300 trajectories for each technique, and the adverse action values based on the states in the visited trajectories and the generated subgoals are reported for clear visualization. Each trajectory corresponds to an action value map that visualizes the action value in each state in the environment. The maximum reported values among the 300 trajectories at each position are illustrated to highlight the differences between our HVSG, HER (Andrychowicz et al., 2017), and HVSG_CVAE, which refers to the method that utilizes CVAE to design the subgoal generator without considering past achievements.

Fig. 8 visualizes the negative action values of the states and sampled subgoals, facilitating clear visualization. The states in the visited trajectories are labeled with negative action values in light colors, while the other regions are visualized with zero scalar values in dark colors. The color bar illustrates that areas with lighter colors correspond to higher reported values, indicating obstacles that require more effort to overcome. The results depicted in Fig. 8 demonstrate that our HVSG method learns the positions of obstacles earlier, specifically in the 25th and 30th training epochs, compared to HER (Andrychowicz et al., 2017). This indicates that our method enables the policy to learn obstacle positions effectively compared to the baselines, leading to more effective goal-conditioned action value learning. In the later training stages, the reported values based on the visited states in our HVSG remain higher than those in HER. For instance, the reported negative action values are 0.80 and 0.76 in HVSG, compared to 0.85 and 0.84 in HER (Andrychowicz et al., 2017) in the 50th and 55th epochs. These results suggest that the action values of -0.80 and -0.76 in HVSG are higher than those of -0.85 and -0.84 in HER, indicating that our HVSG method outperforms the baseline HER (Andrychowicz et al., 2017).

Furthermore, the action value map shows that HVSG_CVAE is not as effective in capturing the positions of obstacles in the early training epochs, such as the 25th and 30th epochs, compared to HVSG. Additionally, HVSG outperforms HVSG_CVAE in the later stages, exhibiting higher action values. For example, in the 50th and 55th training epochs, the action values are -0.8 and -0.76 in HVSG, compared to -0.88 and -0.9 in HVSG_CVAE. Overall, the reported action value map demonstrates that our HVSG method effectively learns the environment, enabling the generation of highly valued subgoals for efficient action value learning.

5.5. Context variable investigation

The context variable based subgoal generation facilitates the generation of dynamically adjusted and goal-relevant subgoals, enhancing

the efficiency and effectiveness of goal-conditioned policy training. In Fig. 9, visited trajectories are presented to examine the working mechanism of the context variable. The top line shows the trajectories of GCSG_FULL, where the context variable is not used. Specifically, the subgoal generator is replaced with a network with three fully connected layers in GCSG. The bottom line represents the trajectories of our GCSG built based on the context variable. The reported trajectories indicate that the generated subgoals in GCSG_FULL are unsuitable for efficient goal-conditioned action value learning. However, a context variable-guided GCSG enables the generation of subgoals that gradually reach the desired goal. Specifically, the generated subgoals move away from the current state with lower relation during the training stages in GCSG_FULL. For example, the distances between the states and the generated subgoals are reduced from 10.43 to 4.6 gradually in our GCSG, compared with 13.92 to 10.58 for GCSG_FULL in the policy training stages. These reported distances suggest that our method, built based on the context variable, can generate highly valued subgoals to guide the agent to reach the desired goal gradually.

5.6. Regularized value investigation

Our method generates highly valued subgoals based on the regularized action values in different settings, ensuring that the subgoals are relatively easy and easy enough. To further investigate the regularization term's working mechanism in our method, we present the visited trajectories and the generated subgoals in the navigation task. In the visualization, the red star represents the initial position, and the green cross represents the desired goal. The blue line represents the visited trajectories, while the green dots represent the generated subgoals. Each generated subgoal is labeled with a number. It is important to note that some of the generated subgoals may be located in barriers, and the generated goals around the barriers can also be used for goal-conditioned action value learning. The mean distances between the states and the generated goals are reported in brackets for easy discussion.

Fig. 10 reports the visited trajectories and the generated subgoals in GCSG, GCSGMax, GCSGMin, HVSG. GCSGMax maximizing the action values based on the generated subgoals enforces the subgoals being close to the current states. For example, the average distance between the states and the generated goals is 1.62 in GCSGMax compared with GCSG of 6.28 in the 50th training epoch. However, the generated subgoals are too close to the current states in GCSGMax. The maximized value-derived subgoals are located around the current states, which increases the percentage of meaningless steps when reaching the desired goal. GCSGMin also suffers from the issue that the generated subgoals are far away from the current states suffering from the issue of sparse

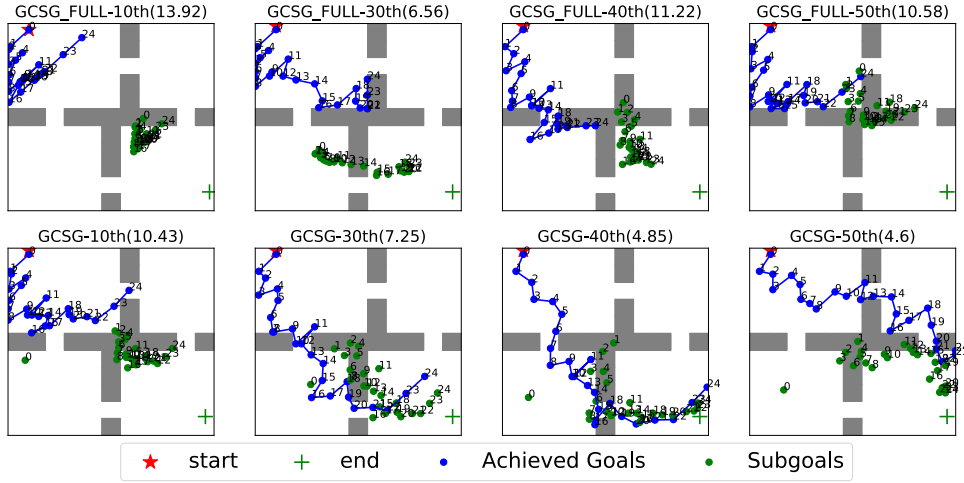


Fig. 9. The visualization of visited trajectories with respect to the context variable.

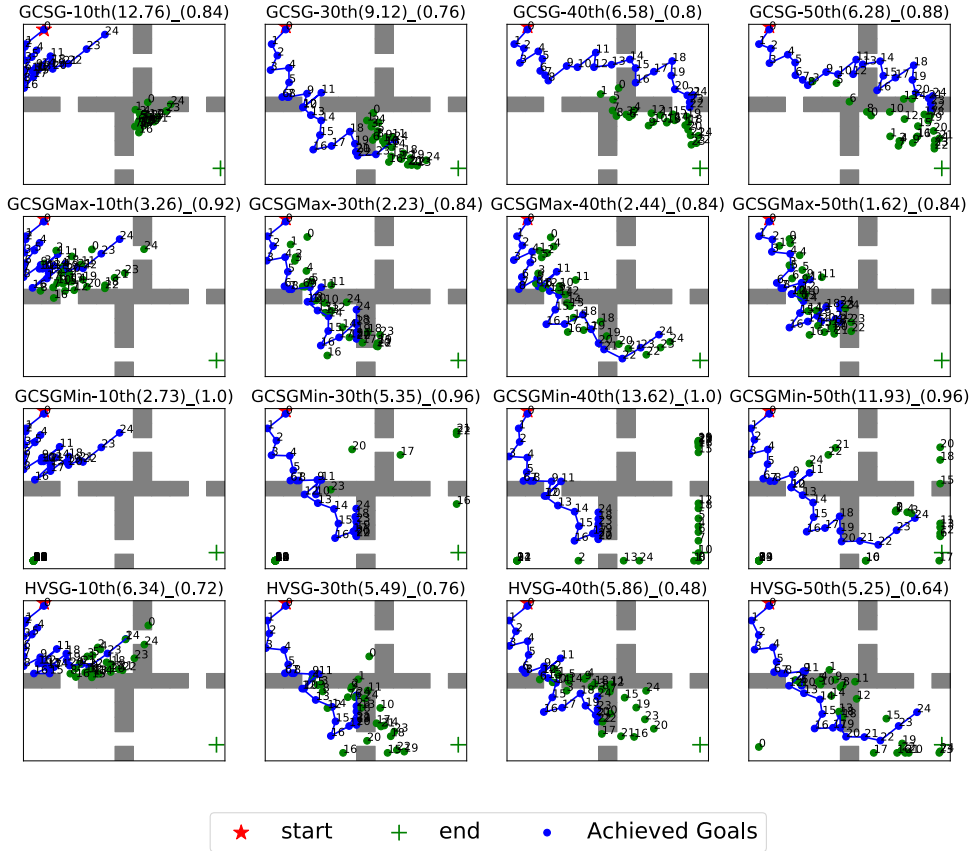


Fig. 10. The visualization of the visited trajectories and the subgoals in different settings of our method, involving GCSG, GCSGMax, GCSGMin, HVSG. The average distances between the achieved goals and the sampled subgoals are reported in the first bracket, while the latter shows the percentage of collision-free subgoals within the obstacle range.

rewards. For example, the average distance between the states and the generated goals is 11.93 of GCSGMin compared with GCSG of 6.28 in the 50th training epoch. Our method alleviates these dilemmas by regularizing the action values in the specific ranges to generate highly valued subgoals. For example, the average distance between the states and the generated goals is 5.25 compared with GCSGMax of 1.62 and GCSGMin of 11.93 in the 50th training epoch.

Fig. 11 reports the visited trajectories and the generated subgoals in the compared methods: RIG (Nair et al., 2018), LEAP (Nasiriany et al., 2019b), RIS (Chane-Sane et al., 2021), HVSG_FULL. RIG (Nair et al., 2018) samples the subgoals only conditioned on the latent variable,

leading to uniformly sampled subgoals from the goal space. There is no mapping between the states and the subgoals in RIG (Nair et al., 2018), and the distance between the states and the subgoals is not reported. LEAP (Nasiriany et al., 2019b) samples the subgoals conditioned on the states and latent variable and maximizes the action value to sample goals, resulting in randomly located subgoals around the obstacles. However, LEAP generates the subgoals without considering the desired goal, which cannot effectively guide the agent to accomplish the tasks. RIS (Chane-Sane et al., 2021) samples the subgoals by maximizing the action values, resulting in generated goals around the states as GCSGMax. Besides, the percentage of collision-free generated goals

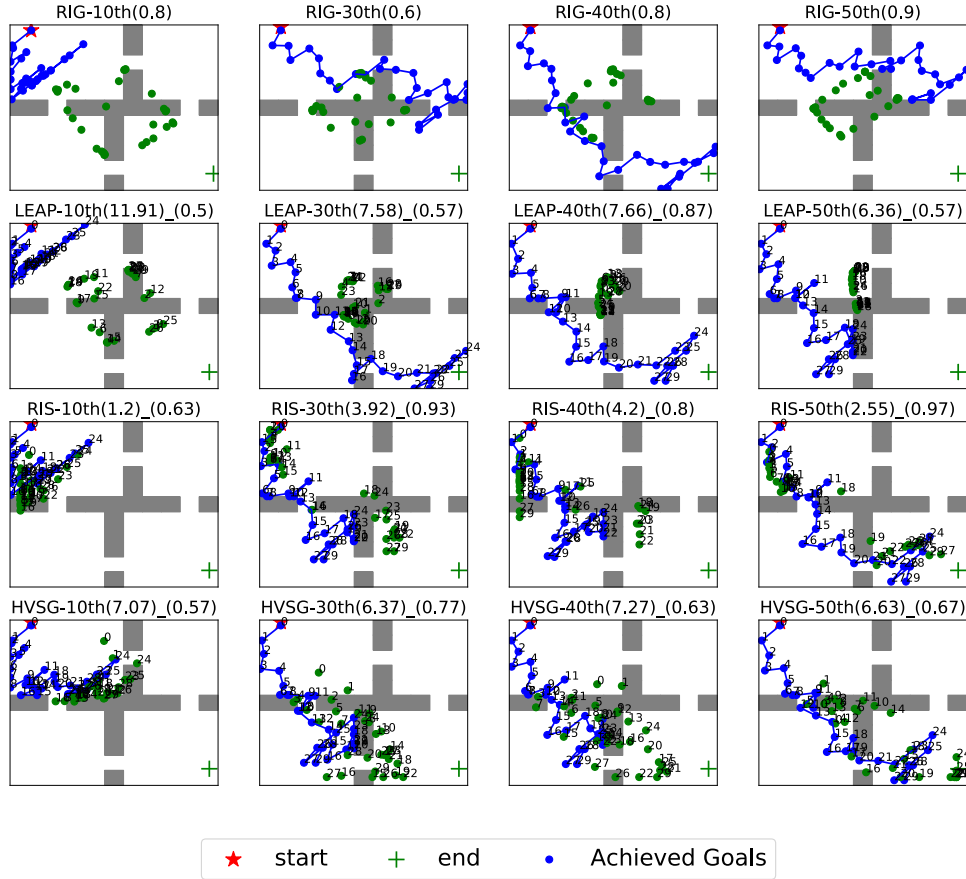


Fig. 11. The visualization includes visited trajectories and subgoals under different baselines, involving the baseline methods LEAP, RIS, and RIG. The average distance between achieved goals and sampled subgoals is reported in the first bracket, while the latter shows the percentage of collision-free subgoals within the obstacle range.

is also reported in the second bracket of the title. The reasonable percent of collision-free instances facilitated by our method contributes to stable policy training. However, it does not solely determine the evaluation performance. The results demonstrate that our method achieves comparable collision-free rates with the baseline. For example, the collision-free rates in our HVSG¹ are 0.63 and 0.67, while in RIS, they are 0.8 and 0.97 in the 40th and 50th training epochs.

Figs. 10 and 11 show that the generated subgoals in our method are more suitable for reaching the desired destination. Specifically, the generated subgoals keep a larger distance from the state for exploration in the initial stage, and these distances are reduced gradually for exploitation in the later stage. The comparing results on the generated subgoals in different settings show the effectiveness of our proposed subgoal-generating method.

5.7. Trajectories visualization in AntMaze

We also compared our method with the method HIRO (Nachum et al., 2018) on the challenging AntMaze task. AntMaze requires the Ant to move from the bottom left corner to the top left corner, covering coordinates from (0, 0) to (16, 16). This task has a maximum step limit of 500, while the maximum step limit in PointEnv is 50. In HIRO (Nachum et al., 2018), the high-level policy is trained using sparse rewards obtained from the environment. In contrast, the low-level policy is trained based on the negative distances between the states and the generated subgoals.

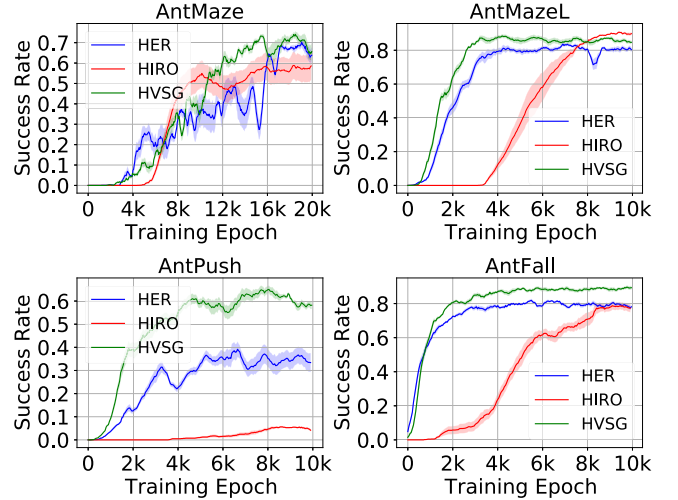


Fig. 12. The evaluation results in Ant-based goal-conditioned tasks compared with hierarchical reinforcement learning HIRO (Nachum et al., 2018).

Fig. 12 presents the evaluation results for Ant-based control tasks, including AntMaze, AntMazeL, AntFall, and AntPush. The evaluated success rate demonstrates that our HVSG achieves the desired goals with higher sample efficiency than HIRO (Nachum et al., 2018). This can be attributed to the fact that the subgoals in HIRO are sampled from the action space to design the high-level policy in the hierarchical framework, impacting sample efficiency. For instance, the dimension of the action space is 12, and the goal space is only 2. Furthermore,

¹ The different distance and collision-free in Figs. 10 and 11 result from the randomness of the environment.

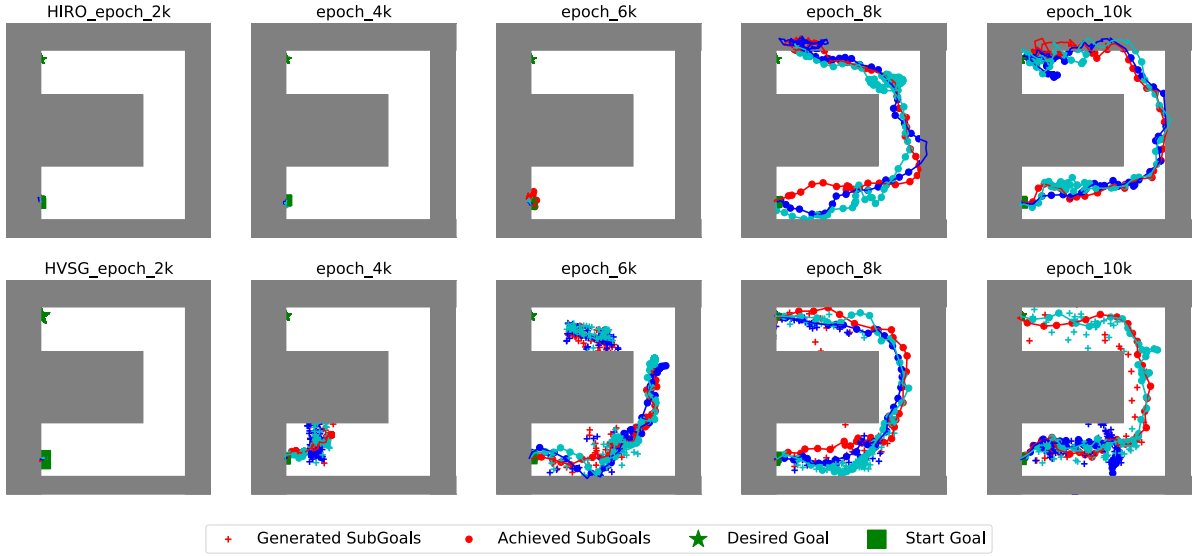


Fig. 13. The visited trajectories in Ant-based goal-conditioned tasks compared with hierarchical reinforcement learning HIRO (Nachum et al., 2018).

our method outperforms HER (Andrychowicz et al., 2017) in terms of sample efficiency and success rate, highlighting the effectiveness of our proposed subgoal generation method. The success rate compared with HER (Andrychowicz et al., 2017) and HIRO (Nachum et al., 2018) demonstrates the superiority of our proposed subgoal-generating method.

The visited trajectories and the generated subgoals were reported in AntMaze in bottom Fig. 13. The subgoals in HIRO (Nachum et al., 2018) were not reported because the subgoals in high-level policy are sampled from the action space, which cannot be visualized. The visited trajectories show that our method reaches the desired goal compared with HIRO (Nachum et al., 2018). For example, most of the finally achieved goals in our HVSG are moving toward the desired goal in the early training stages, but HIRO (Nachum et al., 2018) does not. Specifically, our method rolls out the trajectories toward the desired goal in the 4k and 6k training epochs, but HIRO does not. This phenomenon demonstrates the effectiveness of our subgoal generation method, which is characterized by higher sample efficiency compared with HIRO (Nachum et al., 2018).

5.8. Ablation studies

In this section, the ablation studies on success rates were reported in the MuJoCo tasks to investigate the contributions of our proposed modules involving methods GCSG, GCSGMin, GCSGMinClip, GCSGMax, and GCSGMaxClip, respectively.

Fig. 14 reports the success rates on three settings to investigate our proposed methods. **First**, the method GCSG that generates the subgoals conditioned on the desired goal by self-learning was reported. GCSG outperforms HER in some environments, such as PointEnv and Pushing, which shows the high efficiency of the desired goal-conditioned subgoal sampling mechanism. **Second**, compared with GCSGMin and GCSGMinClip, GCSGMax and GCSGMaxClip, results show that the methods with regularized action values are superior to the methods without regularized values in most environments such as FetchPush and PickAndPlace. The performance of GCSGMin shows the worst performance in most environments because the generated subgoals are too hard to provide informative knowledge for policy learning. The comparing results indicate that the generated subgoals with regularized values are characterized by suitable complexity for reaching. **Third**, HVSG combining GCSGMaxClip and GCSGMinClip achieves the most stable performance compared with the baselines. Interestingly, GCSGMaxClip and GCSGMinClip show bad performance compared with HER. However, HVSG

integrating GCSGMaxClip and GCSGMinClip achieves the most stable performance compared with other counterparts. The evaluation results on GCSGMaxClip, GCSGMinClip, and HVSG show the effectiveness of our proposed regularization term for subgoal generating. **Fourth**, the subgoal generator in our HVSG is implemented by conditional VAE coupled with LSTM. This combination is replaced by full connection layers referred to as HVSG_FULL to investigate the effectiveness of our subgoal generator. Compared with HVSG and HVSG_FULL, results show that HVSG_FULL achieves lower success rates in the compared methods. This phenomenon suggests that the full connection layer based generator is unsuitable for subgoal generation. We also compared our method with HVSG_CVAE without using LSTM to design our subgoal generator, in which the latent variable cannot capture the latent representation of past achievements. Results show that HVSG_CVAE is inferior to our method, which shows the effectiveness of the latent representation based on the past achieved goals for subgoal generation. The comparisons of the four settings show the high efficiency of our proposed method in goal-conditioned reinforcement learning.

Additionally, we investigated the distances between the sampled subgoals and the states for subgoal generator training. The subgoal generator is trained based on the visited trajectories, involving the sampling of the future achieved goal $\hat{g}'_t = \phi(s_{t+k})$, $k \geq 1$. The distances between the sampled goals $\hat{g}'_t$ and the current states determine the quality of the generated subgoals. The future offset k is a random variable sampled from the specific range $[1, T - t]$. To investigate the effect of different k values on our method, we set different upper bounds k_u for k sampling by replacing $T - t$ with $k_u = (1, 3, 5, 10, 25, 40)$. We used these values to examine the effect of k on the success rate, as reported in Fig. 14. The evaluation results indicate that success rates generally increase with a larger range in most tasks. This phenomenon arises because smaller k_u correspond to subgoals with shorter distances between states, which negatively affects exploration for action value learning. Conversely, larger k mitigate this issue, enabling more efficient exploration.

We also conducted further ablation studies to evaluate the robustness of our method under different hyper-parameter α settings in the second line of Fig. 14. In the classical Fetch environment, we tested hyper-parameter values of 1, 0.1, 0.01, 0.001, and 0.0001 to analyze the stability of our method in Fetch. The experimental analysis indicates that our proposed method maintains stable performance with $\alpha = 0.001$. However, PointEnv achieves the most stable performance across various hyper-parameters. This discrepancy is attributed to the differing feasible goal space ranges in the two environments; the goal

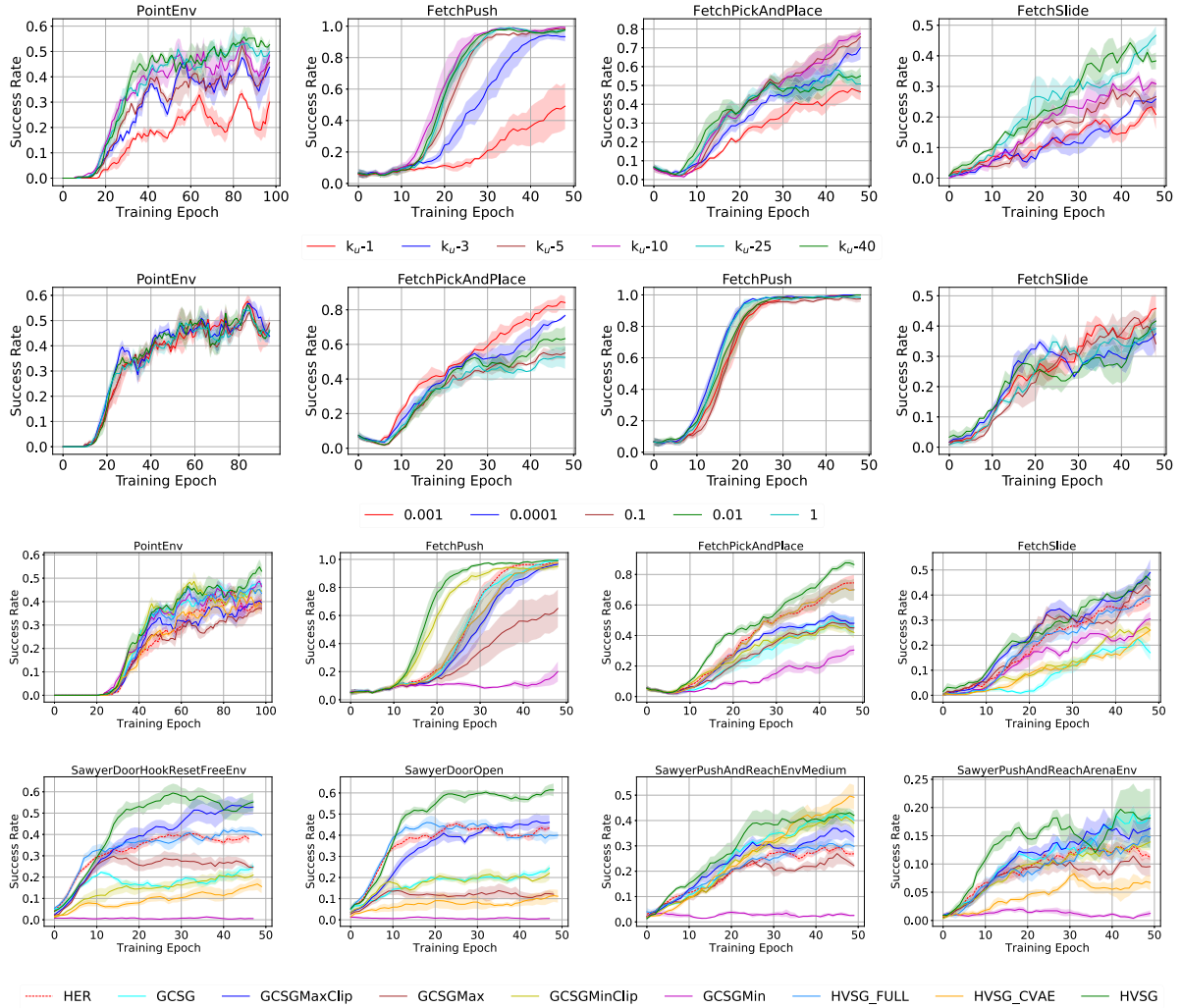


Fig. 14. The ablation studies in our method with five different seeds.

space in the Fetch task is relatively small, ranging from 0.01 to 0.05, whereas the goal space in PointEnv is considerably larger, ranging from 0 to 1. Specifically, the smaller goal space in Fetch requires precise tuning of the update steps of the subgoal generator, corresponding to a smaller hyper-parameter, while the larger goal space in PointEnv allows for the generator to update with larger steps at any value. The evaluation results demonstrate that our method is robust with respect to the selection of hyper-parameter values.

6. Conclusions

This paper proposes a subgoal sampling method trained based on self-designed demonstrations. It generates subgoals conditioned on the context of the tasks and the desired goal for goal-conditioned policy training. Furthermore, the regularized action value is also used to generate subgoals with suitable complexity for different training stages. However, our method is time-consuming due to the sequential subgoal generation compared to the baseline methods. Experimental analysis in the navigation task shows that the generated subgoals, conditioned on the task context and the desired goal, guide the agent to reach the desired goal effectively. Additionally, the subgoals are suitable for achieving the goal compared to HER, as visualized through the action value map in different training stages. Furthermore, the evaluation results demonstrate that our method achieves the most stable performance compared to the baseline methods. In the future, we plan to develop a more adaptable and versatile method for measuring

the distances between states and goals in complex tasks to facilitate subgoal generation, which can be used in various fields such as intelligent automation (Zhang et al., 2021), robotics (Santucci et al., 2016), smart manufacturing systems (Bai et al., 2023; Bai, Li, & Tong, 2020), adaptive learning environments, and advanced decision-making processes (Devidze et al., 2022; Fu et al., 2022; Sivamayil et al., 2023) in dynamic and uncertain environments.

CRedit authorship contribution statement

Yao Li: Writing – review & editing, Writing – original draft, Methodology. **YuHui Wang:** Writing – original draft. The majority of the work was completed before YuHui joined KAUST. **XiaoYang Tan:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by National Science Foundation of China (6247072715).

Data availability

No data was used for the research described in the article.

References

- Andrychowicz, M., Crow, D., Ray, A., Schneider, J., Fong, R., Welinder, P., et al. (2017). Hindsight experience replay. In *Proceedings of the 30th conference on neural information processing systems, neurIPS* (pp. 5048–5058).
- Bai, W., Li, T., Long, Y., & Chen, C. L. P. (2023). Event-triggered multigradient recursive reinforcement learning tracking control for multiagent systems. *IEEE Transactions on Neural Networks and Learning Systems*, 34(1), 366–379.
- Bai, W., Li, T., & Tong, S. (2020). NN reinforcement learning adaptive control for a class of nonstrict-feedback discrete-time systems. *IEEE Transactions on Cybernetics*, 50(11), 4573–4584.
- Bai, C., Wang, L., Wang, Y., Wang, Z., Zhao, R., Bai, C., et al. (2023). Addressing hindsight bias in multi-goal reinforcement learning. *IEEE Transactions on Cybernetics*, 53(1), 392–405.
- Bing, Z., Álvarez, E., Cheng, L., Morin, F. O., Li, R., Su, X., et al. (2021). Robotic manipulation in dynamic scenarios via bounding-box-based hindsight goal generation. *IEEE Transactions on Neural Networks and Learning Systems*, 1–14.
- Bing, Z., Brucker, M., Morin, F. O., Li, R., Su, X., Huang, K., et al. (2022). Complex robotic manipulation via graph-based hindsight goal generation. *IEEE Transactions on Neural Networks and Learning Systems*, 33(12), 7863–7876.
- Bramlage, L., & Cortese, A. (2022). Generalized attention-weighted reinforcement learning. *Neural Networks*, 145, 10–21.
- Chandrasekar, A., Radhika, T., & Zhu, Q. (2022a). Further results on input-to-state stability of stochastic Cohen–Grossberg BAM neural networks with probabilistic time-varying delays. *Neural Processing Letters*, 1–23.
- Chandrasekar, A., Radhika, T., & Zhu, Q. (2022b). State estimation for genetic regulatory networks with two delay components by using second-order reciprocally convex approach. *Neural Processing Letters*, 54(1), 327–345.
- Chandrasekar, A., Rakkiyappan, R., & Li, X. (2016). Effects of bounded and unbounded leakage time-varying delays in memristor-based recurrent neural networks with different memductance functions. *Neurocomputing*, 202, 67–83.
- Chane-Sane, E., Schmid, C., & Laptev, I. (2021). Goal-conditioned reinforcement learning with imagined subgoals. In *Proceedings of the 38th international conference on machine learning, ICML 2021, 18–24 July 2021, virtual event: vol. 139*, (pp. 1430–1440).
- Devidze, R., Kamalaruban, P., & Singla, A. (2022). Exploration-guided reward shaping for reinforcement learning under sparse rewards. In *Proceedings of the 35th neural information processing systems, neurIPS 2022*.
- Fang, M., Zhou, T., Du, Y., Han, L., & Zhang, Z. (2019). Curriculum-guided hindsight experience replay. In *Proceedings of the 32nd conference on neural information processing systems, neurIPS* (pp. 12602–12613).
- Florensa, C., Held, D., Geng, X., & Abbeel, P. (2018). Automatic goal generation for reinforcement learning agents. In *Proceedings of the 35th international conference on machine learning, ICML, 2018: vol. 80*, (pp. 1515–1528).
- Fu, Q., Han, Z., Chen, J., Lu, Y., Wu, H., & Wang, Y. (2022). Applications of reinforcement learning for building energy efficiency control: A review. *Journal of Building Engineering*, [ISSN: 2352-7102] 50, Article 104165.
- Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In J. G. Dy, & A. Krause (Eds.), *Proceedings of the 35th international conference on machine learning, ICML, 2018: vol. 80*, (pp. 1582–1591).
- Huang, C., Zhang, R., Ouyang, M., Wei, P., Lin, J., Su, J., et al. (2021). Deductive reinforcement learning for visual autonomous urban driving navigation. *IEEE Transactions on Neural Networks and Learning Systems*, 32(12), 5379–5391.
- Kumar, G., Shankar, N. S., Didwania, H., Roychoudhury, R. D., Bhowmick, B., & Krishna, K. M. (2021). GCExp: Goal-conditioned exploration for object goal navigation. In *Proceedings of the 30th IEEE international conference on robot & human interactive communication, RO-MAN 2021* (pp. 123–130). IEEE.
- Lai, Y., Wang, W., Yang, Y., Zhu, J., & Kuang, M. (2020). Hindsight planner. In *Proceedings of the 19th international conference on autonomous agents and multiagent systems, AAMAS '20* (pp. 690–698).
- Li, M., Huang, T., & Zhu, W. (2022). Clustering experience replay for the effective exploitation in reinforcement learning. *Pattern Recognition*, 131, Article 108875.
- Li, H., Luo, B., Song, W., & Yang, C. (2023). Predictive hierarchical reinforcement learning for path-efficient mapless navigation with moving target. *Neural Networks*, 165, 677–688.
- Li, Y., Wang, Y., Gan, Y., & Tan, X. (2022). Alleviating the estimation bias of deep deterministic policy gradient via co-regularization. *Pattern Recognition*, 131, Article 108872.
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., et al. (2016). Continuous control with deep reinforcement learning. In *Proceedings of 4th international conference on learning representations, ICLR, 2016, san juan, puerto rico, May 2–4, 2016*.
- Liu, H., Trott, A., Socher, R., & Xiong, C. (2019). Competitive experience replay. In *Proceedings of the 7th international conference on learning representations, ICLR 2019*.
- Ming, F., Gao, F., Liu, K., & Zhao, C. (2023). Cooperative modular reinforcement learning for large discrete action space problem. *Neural Networks*, 161, 281–296.
- Nachum, O., Gu, S., Lee, H., & Levine, S. (2018). Data-efficient hierarchical reinforcement learning. In *Proceedings of the 32nd neural information processing systems, neurIPS 2018* (pp. 3307–3317).
- Nair, A., Pong, V., Dalal, M., Bahl, S., Lin, S., & Levine, S. (2018). Visual reinforcement learning with imagined goals. In *Proceedings of the 31st conference on neural information processing systems, neurIPS* (pp. 9209–9220).
- Nasiriany, S., Pong, V., Lin, S., & Levine, S. (2019a). Planning with goal-conditioned policies. In *Proceedings of the 33rd neural information processing systems, neurIPS* (pp. 14814–14825).
- Nasiriany, S., Pong, V., Lin, S., & Levine, S. (2019b). Planning with goal-conditioned policies. In *Proceedings of the 32nd neural information processing systems, neurIPS 2019* (pp. 14814–14825).
- Pong, V., Dalal, M., Lin, S., Nair, A., Bahl, S., & Levine, S. (2020). Skew-fit: State-covering self-supervised reinforcement learning. In *Proceedings of the 37th international conference on machine learning, ICML, 13–18 July 2020, virtual event: vol. 119*, (pp. 7783–7792).
- Radhika, T., Chandrasekar, A., Vijayakumar, V., & Zhu, Q. (2023). Analysis of Markovian jump stochastic Cohen–Grossberg BAM neural networks with time delays for exponential input-to-state stability. *Neural Processing Letters*, 55(8), 11055–11072.
- Rakkiyappan, R., Chandrasekar, A., & Cao, J. (2014). Passivity and passification of memristor-based recurrent neural networks with additive time-varying delays. *IEEE Transactions on Neural Networks and Learning Systems*, 26(9), 2043–2057.
- Rao, Z., Wu, Y., Yang, Z., Zhang, W., Lu, S., Lu, W., et al. (2021). Visual navigation with multiple goals based on deep reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 32(12), 5445–5455.
- Santucci, V. G., Baldassarre, G., & Mirolli, M. (2016). GRAIL: A goal-discovering robotic architecture for intrinsically-motivated learning. *IEEE Transactions on Cognitive and Developmental Systems*, 8(3), 214–231.
- Sivamayil, K., Rajasekar, E., Aljafari, B., Nikolovski, S., Vairavasundaram, S., & Vairavasundaram, I. (2023). A systematic study on reinforcement learning based applications. *Energies*, 16(3), 1512.
- Sohn, S., Lee, S., Choi, J., van Seijen, H., Fatemi, M., & Lee, H. (2021). Shortest-path constrained reinforcement learning for sparse reward tasks. In *Proceedings of the 38th international conference on machine learning, ICML 2021, 18–24 July 2021, virtual event: vol. 139*, (pp. 9780–9790).
- Sohn, K., Lee, H., & Yan, X. (2015). Learning structured output representation using deep conditional generative models. In *Proceedings of the 28th conference on neural information processing systems, neurIPS* (pp. 3483–3491).
- Tamil Thendral, M., Ganesh Babu, T. R., Chandrasekar, A., & Cao, Y. (2022). Synchronization of Markovian jump neural networks for sampled data control systems with additive delay components: Analysis of image encryption technique. *Mathematical Methods in the Applied Sciences*.
- Tsantekidis, A., Passalis, N., & Tefas, A. (2023). Modeling limit order trading with a continuous action policy for deep reinforcement learning. *Neural Networks*, 165, 506–515.
- Vecchiotti, L. F., Seo, M., & Har, D. (2022). Sampling rate decay in hindsight experience replay for robot control. *IEEE Transactions on Cybernetics*, 52(3), 1515–1526.
- Vezhnevets, A. S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., et al. (2017). Feudal networks for hierarchical reinforcement learning. In *Proceedings of the 34th international conference on machine learning, ICML 2017: vol. 70*, sydney, NSW, Australia, 6–11 August 2017, (pp. 3540–3549).
- Yang, R., Lu, Y., Li, W., Sun, H., Fang, M., Du, Y., et al. (2022). Rethinking goal-conditioned supervised learning and its connection to offline RL. In *Proceedings of the tenth international conference on learning representations, ICLR 2022, virtual event, April 25–29, 2022*.
- Zhang, Z., Gan, Y., & Tan, X. (2022). Robust action gap increasing with clipped advantage learning. In *Proceedings of the 36th conference on artificial intelligence, AAAI 2022, 2022 virtual event, February 22 – March 1, 2022* (pp. 9145–9152).
- Zhang, T., Guo, S., Tan, T., Hu, X., & Chen, F. (2020). Generating adjacency-constrained subgoals in hierarchical reinforcement learning. In *Proceedings of the 33rd neural information processing systems, neurIPS 2020, December 6–12, 2020, virtual*.
- Zhang, Z., Wang, D., & Gao, J. (2021). Learning automata-based multiagent reinforcement learning for optimization of cooperative tasks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(10), 4639–4652.
- Zu, L., He, X., Yang, J., Liu, L., & Wang, W. (2023). A reinforcement learning algorithm acquires demonstration from the training agent by dividing the task space. *Neural Networks*, 164, 419–427.